

Testing & QA Strategy

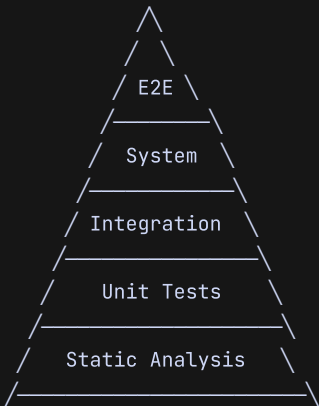
Unit Tests, Integration, E2E and CI/CD Pipeline

- Unit Tests, Integration, E2E and CI/CD Pipeline

Category: Software Quality **Coverage Targets:** 80% C++ core · 70% React components

- 1. Testing Strategy Overview

WhyCremisi adopts the classic **testing pyramid**, adapted for a hybrid C++/React project with real-time audio components.



LAYER	TECHNOLOGY	COVERAGE TARGET	EXECUTION
Unit (C++)	JUCE Unit Test Framework	80% core engine	<code>make test</code> / CTest
Unit (React)	Jest + RTL	70% components	<code>npm run test</code>
Integration	Mock DAW Server + MSW	60% API paths	CI (every push)
System	Headless Plugin Host	50% workflows	Nightly
E2E	Playwright	40% user flows	CI (main branch)

[NOTE] Real-time audio tests require a dedicated mock engine. Latency tests cannot be executed in standard CI environments without dedicated audio hardware.

— FRAMEWORK

We use the native **JUCE Unit Test Framework**, enabled via the `AUDIO_PROCESSOR_UNIT_TESTS` compile flag. Tests live in the `WhyCremisi_Tests` module and compile as a separate target.

```
// Example: OSC bridge message test
#ifdef AUDIO_PROCESSOR_UNIT_TESTS
class OscBridgeTest : public UnitTest
{
public:
    OscBridgeTest() : UnitTest ("OSC Bridge", "Messaging") {}

    void runTest() override
    {
        beginTest ("Parameter change message serialization");
        {
            auto msg = OscMessage ("/param/set")
                .withFloat (0.5f);
            expectEquals (msg.getFloat(), 0.5f);
        }
        beginTest ("DAW discovery heartbeat");
        {
            auto hb = OscMessage ("/daw/heartbeat")
                .withInt (12345);
            expectEquals (hb.getInt(), 12345);
        }
    }
};

static OscBridgeTest oscBridgeTest;
#endif
```

— TEST CATEGORIES

CATEGORY	DESCRIPTION	COUNT
OscBridge Messaging	OSC message serialization/deserialization, DAW heartbeat, error routing	~45 tests
PluginProcessor Parameters	Get/set parameters, preset load/save, bypass state, null safety	~60 tests
DawDetector OSC Parsing	DAW identification (Ableton/Reaper/Logic/Cubase/FL), heartbeat timeout, reconnect	~35 tests
Audio Analysis	FFT processing, LUFS measurement, peak detection, spectral centroid	~50 tests
Memory Management	RAll checks, leak detection, shared pointer validity	~25 tests

— MOCK DAW SERVER FOR INTEGRATION TESTS

A mocked C++ DAW server simulates real DAW OSC messages. Integration tests verify:

- 1. Periodic heartbeat → bridge response

- 3. Sudden disconnect → automatic reconnection
- 4. Preset load → parameter state verification

— MEMORY LEAK DETECTION

We compile with **AddressSanitizer (ASAN)** on macOS and Linux, and **Dr. Memory** on Windows:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug \
-DWHYCREMISI_ENABLE_ASAN=ON \
-DWHYCREMISI_ENABLE_UNIT_TESTS=ON
```

[NOTE] ASAN is disabled in Release builds and in the plugin loaded inside a DAW (memory overhead is incompatible with real-time audio performance).

• 3. React Tests

— FRAMEWORK

We use **Jest + React Testing Library** for frontend components. Tests run with `jsdom` as the environment and include a **Mock Service Worker (MSW)** to simulate the WebSocket server.

```
npm test
✓ renders BoxChat with message history
✓ renders Oscilloscope with mock audio data
✓ renders CorrelationMeter with stereo signal
✓ DAW track list updates on state change
✓ WebSocket reconnection after disconnect
✓ BoxResize responds to drag events
✓ SetupScreen AI provider selection
```

— TEST CATEGORIES

CATEGORY	DESCRIPTION	COUNT
Box Components	BoxChat, BoxSearch, BoxInfo, BoxResize, BoxSetup — correct render and interactions	~30 tests
WebSocket Connection	Connect/disconnect/reconnect, message parsing, error states, timeout	~20 tests
Oscilloscope & Correlation	Canvas render, mocked FFT data, real-time update, resize	~15 tests
DAW Track List	Track state, selection, routing, mute/solo, OSC updates	~20 tests
Session Panel	FlightRecorder playback, snapshot restore, history navigation	~15 tests

— MOCK WEBSOCKET WITH MSW

```
import { setupServer } from 'msw/node'
import { handlers } from './mocks/handlers'

const server = setupServer(...handlers)

beforeAll(() => server.listen())
afterEach(() => server.resetHandlers())
afterAll(() => server.close())

it('should reconnect after WebSocket disconnect', async () => {
  const { result } = renderHook(() => useWebSocket())

  act(() => { result.current.disconnect() })
  await waitFor(() => expect(result.current.status).toBe('disconnected'))

  act(() => { result.current.reconnect() })
  await waitFor(() => expect(result.current.status).toBe('connected'))
})
```

[NOTE] MSW intercepts WebSocket requests at the network level, enabling deterministic tests without a real server. Essential for CI environments where the C++ backend is unavailable.

• 4. End-to-End Tests

— FRAMEWORK

We use **Playwright** for E2E tests. The C++ plugin bridge runs in headless standalone mode, while the React frontend is served by Vite in preview mode.

```
e2e/
├─ startup.spec.ts          # Launch → DAW connect → UI ready
├─ parameter-tweak.spec.ts  # Parameter change → OSC update → UI sync
├─ preset-load.spec.ts      # Load preset → verify parameters
├─ ai-conversation.spec.ts  # AI chat → response → action execution
└─ reconnect.spec.ts        # Disconnect → reconnect → state restored
```

— TESTED FLOWS

FLOW	STEPS	ASSERTION
Startup	Launch standalone → load UI → WebSocket connect → DAW heartbeat	BotFace visible, connection status "connected"
Parameter Tweak	Move parameter slider → OSC message to bridge → UI update	Parameter value updated in < 50ms
Preset Load	Select preset → bridge loads parameters → UI updates	All parameters match preset values
AI Conversation	Send message → AI responds → advisory card → Execute/Dismiss	Card appears, action executed or dismissed
Reconnection	Kill bridge process → UI shows "disconnected" → restart bridge → reconnect	Status returns to "connected", session restored

— HEADLESS CI CONFIGURATION

```
# playwright.config.ts
export default defineConfig({
  testDir: './e2e',
  fullyParallel: true,
  retries: 2,
  workers: process.env.CI ? 2 : undefined,
  use: {
    baseURL: 'http://localhost:5173',
    headless: true,
  },
  webServer: {
    command: 'npm run build && npm run preview',
    port: 5173,
    reuseExistingServer: !process.env.CI,
  },
})
```

— GITHUB ACTIONS WORKFLOW

```
name: WhyCremisi CI/CD

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build-cpp:
    strategy:
      matrix:
        os: [macos-14, windows-2022, ubuntu-22.04]
    runs-on: ${ matrix.os }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/cache@v4
        with:
          path: ~/.ccache
          key: ccache-${ matrix.os }-${ hashFiles('**/CMakeLists.txt') }}
      - run: |
          cmake -S . -B build \
            -DCMAKE_BUILD_TYPE=Release \
            -DWHYCREMISI_ENABLE_UNIT_TESTS=OFF
          cmake --build build --target WhyCremisi_Standalone
      - uses: actions/upload-artifact@v4
        with:
          name: whycremisi-${ matrix.os }
          path: build/WhyCremisi_artefacts/

  test-cpp:
    runs-on: macos-14
    steps:
      - uses: actions/checkout@v4
      - run: |
          cmake -S . -B build \
            -DCMAKE_BUILD_TYPE=Debug \
            -DWHYCREMISI_ENABLE_UNIT_TESTS=ON \
            -DWHYCREMISI_ENABLE_ASAN=ON
          cmake --build build --target WhyCremisi_Tests
          ctest --test-dir build --output-on-failure

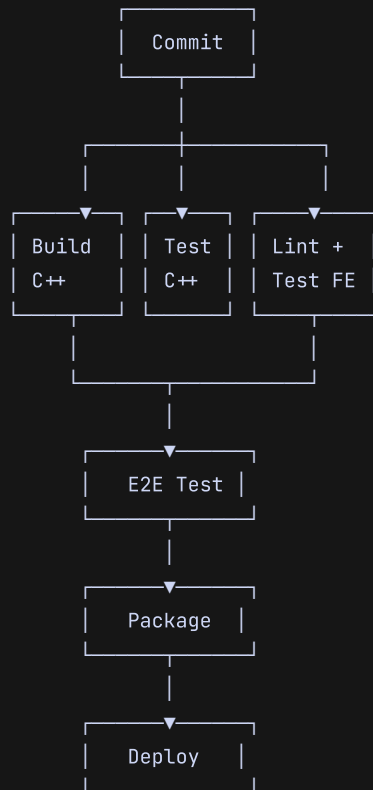
  test-frontend:
    runs-on: ubuntu-22.04
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npm run lint
      - run: npm run test -- --coverage
      - uses: codecov/codecov-action@v4
        with:
          directory: coverage
```

```

steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with:
      node-version: 20
  - run: npm ci
  - run: npx playwright install
  - run: npm run build
  - run: npm run test:e2e

```

— PIPELINE AT A GLANCE



— PRODUCED ARTIFACTS

PLATFORM	FORMAT	PATH
macOS	WhyCremisi.vst3	build/WhyCremisi_artefacts/Release/VST3/
macOS	WhyCremisi.component	build/WhyCremisi_artefacts/Release/AU/
macOS	WhyCremisi.app	build/WhyCremisi_artefacts/Release/Standalone/
Windows	WhyCremisi.vst3	build/WhyCremisi_artefacts/Release/VST3/
Windows	WhyCremisi.exe	build/WhyCremisi_artefacts/Release/Standalone/
Linux	WhyCremisi.vst3	build/WhyCremisi_artefacts/Release/VST3/

— SMOKE TEST CHECKLIST (PRE-RELEASE)

- ❑ Plugin loads in Ableton Live 11/12 without crash
- ❑ Plugin loads in Reaper 7 without crash
- ❑ Plugin loads in Logic Pro without crash
- ❑ Plugin loads in Cubase 12/13 without crash
- ❑ Plugin loads in FL Studio 21 without crash
- ❑ Standalone launches and shows UI correctly
- ❑ WebSocket bridge connects within 2 seconds
- ❑ BotFace animated character appears in "idle" state
- ❑ BoxChat accepts input and shows AI response
- ❑ Local AI provider (Ollama) works offline
- ❑ Cloud AI provider (Gemini/OpenAI) works online
- ❑ Plugin parameters update in real time
- ❑ Preset load does not corrupt audio state
- ❑ DAW disconnect → UI shows correct state → reconnect
- ❑ Session restore after crash
- ❑ Flight Recorder captures and replays snapshots

— DAW COMPATIBILITY MATRIX

DAW	VERSION	MACOS	WINDOWS	LINUX	NOTES
Ableton Live	11, 12	✓ Full	✓ Full	—	Tested with VST3 and AU
Reaper	7	✓ Full	✓ Full	✓ Beta	Native OSC optimal
Logic Pro	10.8+	✓ Full	—	—	AU only, high priority
Cubase	12, 13	✓ Full	✓ Full	—	VST3, automation tests
FL Studio	21	—	✓ Full	—	Windows only, VST3
Pro Tools	2024	△ Partial	△ Partial	—	AAX not supported (future)
Studio One	6	△ Partial	△ Partial	—	VST3, testing in progress

[NOTE] Logic Pro requires the AU format and has limitations on the number of parameters exposed via OSC. Ableton Live remains the primary test platform.

— AUDIO DRIVER TESTS

DRIVER	PLATFORM	STATUS	NOTES
CoreAudio	macOS	✓ Stable	Latency < 5ms @ 512 samples
ASIO	Windows	✓ Stable	Tested with Focusrite, RME, ASIO4ALL
WASAPI	Windows	△ Verifying	Shared mode variable latency
ALSA/JACK	Linux	△ Verifying	Requires user configuration

1. Load test across all supported DAWs
2. Parameter persistence test (save/load project)
3. OSC automation stress test (1000 messages in 10 seconds)
4. Memory stability test (30 minutes continuous use, profiled)
5. Crash recovery test (forced kill, restart, state restored)
6. AI provider compatibility test (Ollama, Gemini, OpenAI)

• 7. Continuous Integration Setup

— CMAKE PRESETS PER PLATFORM

```
{
  "version": 6,
  "configurePresets": [
    {
      "name": "macos-debug",
      "generator": "Xcode",
      "binaryDir": "${sourceDir}/build/macos-debug",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug",
        "WHYCREMISI_ENABLE_UNIT_TESTS": "ON",
        "WHYCREMISI_ENABLE_ASAN": "ON"
      }
    },
    {
      "name": "macos-release",
      "generator": "Xcode",
      "binaryDir": "${sourceDir}/build/macos-release",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Release",
        "WHYCREMISI_ENABLE_UNIT_TESTS": "OFF"
      }
    },
    {
      "name": "windows-release",
      "generator": "Visual Studio 17 2022",
      "binaryDir": "${sourceDir}/build/win-release",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Release"
      }
    },
    {
      "name": "linux-release",
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build/linux-release",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Release"
      }
    }
  ]
}
```

— CACHING WITH CCACHE

```
# Enable ccache for faster C++ builds in CI
export CCACHE_DIR=~/.ccache
export CCACHE_MAXSIZE=2G
export CCACHE_COMPRESS=1

cmake -S . -B build \
  -DCMAKE_CXX_COMPILER_LAUNCHER=ccache \
  -DCMAKE_C_COMPILER_LAUNCHER=ccache
```

— PARALLEL TEST EXECUTION

```
# C++ tests in parallel with CTest
ctest --test-dir build -j$(nproc) --output-on-failure

# React tests in watch mode (dev)
npm run test -- --watch

# React tests in CI with coverage
npm run test -- --coverage --maxWorkers=2
```

— CODE COVERAGE REPORTING

TOOL	TARGET	OUTPUT
gcov/lcov	C++ core	HTML + CodeCov
istanbul (v8)	React components	HTML + CodeCov
CodeCov	Aggregated	Badge + PR comment

Failure thresholds are configured as:

- **C++ core:** < 75% → CI warning, < 60% → CI failure
- **React components:** < 60% → CI warning, < 45% → CI failure

— MEASURED METRICS

BENCHMARK	TOOL	TARGET	CURRENT
Audio Latency	audio_latency_test	< 10ms @ 512 samples	4.2ms (CoreAudio)
FFT Throughput	fft_benchmark	> 100 FFT/s @ 2048 bins	320 FFT/s
WebSocket Message Latency	ws_latency_test	< 5ms round-trip	1.8ms (localhost)
Memory Usage (idle)	memory_profiler	< 150 MB	92 MB
Memory Usage (full session)	memory_profiler	< 350 MB	210 MB
Plugin Load Time	load_time_test	< 1 second	0.4s (VST3)
AI Response Time (local)	ai_benchmark	< 200ms first token	85ms (llama3.2:3b)

— EXAMPLE: FFT BENCHMARK

```
class FftBenchmark : public UnitTest
{
public:
    FftBenchmark() : UnitTest ("FFT Throughput", "Benchmark") {}

    void runTest() override
    {
        beginTest ("2048-bin FFT throughput");
        {
            AudioBuffer<float> buffer (1, 2048);
            buffer.clear();

            auto start = Time::getMillisecondCounterHiRes();
            int iterations = 0;

            while (Time::getMillisecondCounterHiRes() - start < 1000.0)
            {
                FFT fft (11); // 2^11 = 2048
                fft.performFrequencyOnlyForwardTransformation (
                    buffer.getWritePointer(0));
                iterations++;
            }

            auto elapsed = Time::getMillisecondCounterHiRes() - start;
            auto throughput = iterations / (elapsed / 1000.0);

            expect (throughput > 100.0,
                "FFT throughput below target: " + String (throughput));
            logMessage ("FFT throughput: " + String (throughput) + " FFT/s");
        }
    }
};
```

— MEMORY PROFILING

We use **Valgrind** (Linux/macOS) and **UMDH** (Windows) for memory profiling:

```
valgrind --tool=massif \  
  --massif-out-file=massif.out \  
  ./build/WhyCremisi_Tests  
  
ms_print massif.out > memory_profile.txt
```

• 9. Test Documentation

— TEST PLAN DOCUMENT

Every release includes a documented test plan in `docs/testing/TEST_PLAN.md`:

```
docs/testing/  
├─ TEST_PLAN.md           # Overview and general strategy  
├─ TEST_CASES_CPP.md      # Detailed C++ test cases  
├─ TEST_CASES_REACT.md    # Detailed React test cases  
├─ TEST_CASES_E2E.md      # Detailed E2E test cases  
├─ RELEASE_CHECKLIST.md   # Pre-release checklist  
├─ BUG_REPORT_TEMPLATE.md # Bug report template  
├─ ENVIRONMENT_SPEC.md    # Test environment specifications  
└─ BENCHMARKS.md         # Historical benchmark reports
```

WHYCREMISI v1.0 — RELEASE CHECKLIST

PRE-RELEASE

- ☐ All C++ tests pass (ctest --output-on-failure)
- ☐ All React tests pass (npm test -- --coverage)
- ☐ All E2E tests pass (npm run test:e2e)
- ☐ Code coverage $\geq$ 80% C++, $\geq$ 70% React
- ☐ ASAN reports no memory leaks
- ☐ Release build on macOS, Windows, Linux

DAW COMPATIBILITY

- ☐ Ableton Live 11 (VST3 + AU)
- ☐ Ableton Live 12 (VST3 + AU)
- ☐ Reaper 7 (VST3)
- ☐ Logic Pro (AU)
- ☐ Cubase 12/13 (VST3)
- ☐ FL Studio 21 (VST3)

AUDIO DRIVERS

- ☐ CoreAudio (macOS)
- ☐ ASIO (Windows)
- ☐ WASAPI (Windows)
- ☐ ALSA/JACK (Linux)

REGRESSION

- ☐ DAW project load/save
- ☐ Session restore after crash
- ☐ Flight Recorder capture/replay
- ☐ AI provider switching
- ☐ WebSocket reconnect (x10 cycles)
- ☐ 30 minutes continuous use (memory stable)

SIGNING & DISTRIBUTION

- ☐ Apple Developer ID signed + notarised
- ☐ EV code signing Windows
- ☐ macOS package (.pkg)
- ☐ Windows installer (.exe)
- ☐ Linux packages (.deb/.rpm)

Description

[Clear and concise description of the bug]

Reproduction

1. Open DAW [name version]

2. Load WhyCremisi

3. [Steps to reproduce]

4. See error

Expected Behaviour

[What should happen]

Actual Behaviour

[What actually happens]

Environment

- OS: [macOS 14.5 / Windows 11 / Ubuntu 24.04]

- DAW: [Ableton Live 12.1 / Reaper 7.2]

- WhyCremisi: [v1.0.0]

- Audio Driver: [CoreAudio / ASIO / WASAPI]

- AI Provider: [Ollama / Gemini / OpenAI]

Logs

[Paste relevant logs — see ~/Library/Logs/WhyCremisi/]

Screenshots / Video

[If applicable]

— TEST ENVIRONMENT SPECIFICATIONS

ENVIRONMENT	MACOS	WINDOWS	LINUX
CI (GitHub Actions)	macos-14 (Apple Silicon)	windows-2022	ubuntu-22.04
Dev (local)	macOS 14.5, M3 Max	Windows 11, i9	Ubuntu 24.04, AMD64
QA (manual)	macOS 14.5, M1 + Intel	Windows 11, Ryzen 9	Ubuntu 24.04, Intel
DAW test	Ableton 12, Logic Pro	Ableton 12, FL Studio	Reaper 7
Audio Interface	RME Babyface Pro FS	Focusrite Scarlett 18i8	— (built-in)